

Vorbereitungstag: Projekt 1

Datenanwendung mit Python – Werkzeuge, Vorgehen, Erwartungen

Erik Schwob

Robotron Bildungszentrum Halle

Tag 0

Heute: Der Werkzeugkasten

Sieben Tage Projektarbeit beginnen morgen.

Heute schauen wir uns gemeinsam an, mit welchen Werkzeugen Sie arbeiten werden.

Tag 0: Vorbereitungstag Projekt 1
Datenanwendung mit Python
– der Werkzeugkasten

Lernziele

- Sie kennen die zwei Themenvarianten und können wählen
- Sie haben einen Überblick über den Tech-Stack
- Sie wissen, was eine Anforderungsanalyse ist und wie eine Projektstruktur aussieht
- Sie kennen die Spielregeln zum Umgang mit KI-Werkzeugen
- Sie haben Ihre Entwicklungsumgebung einsatzbereit

- ➊ Projekt im Überblick und die zwei Varianten
- ➋ Vorgehen – Anforderungsanalyse und Projektstruktur
- ➌ Tech-Stack-Tour
 - Frontend-Optionen
 - Datenhaltung
 - Externe APIs
 - Datenanalyse mit pandas
- ➍ Querschnittsthemen – Datenschutz, Fehlerbehandlung, Geheimnisse
- ➎ KI-Werkzeuge – erlaubt, erwünscht, kritisch
- ➏ Organisatorisches und Abgabe
- ➐ Setup und Aufwärmen

Eine Datenanwendung

Sie bauen ein Programm, das

- Daten von einer **externen Quelle** abrufen (API),
- sie mit **eigenen Daten** kombiniert (Datenbank/Excel),
- sie **auswertet** (pandas),
- das Ergebnis in einer **bedienbaren Oberfläche** präsentiert.

Was Sie lernen

Den vollständigen Entwicklungsprozess – von der Idee bis zur Übergabe.
Nicht jedes einzelne Werkzeug perfekt, sondern das Zusammenspiel.

Variante A

Strompreis-Assistent

Aktuelle Börsenstrompreise abrufen,
Verbraucher verwalten, günstige
Zeitfenster empfehlen.

API: aWATTar

Daten: Verbraucher, Preise,
Empfehlungen

Variante B

Reise- und Städte-Dashboard

Wunschstädte verwalten, Wetter
abrufen, Klima vergleichen.

API: Open-Meteo (+ REST
Countries optional)

Daten: Städte, Wetter-Historie

Beide Varianten sind gleichwertig und gleich anspruchsvoll. Wählen Sie nach Interesse.

Merksatz: Egal welche Variante: Ihre Anwendung muss diese sechs Punkte erfüllen.

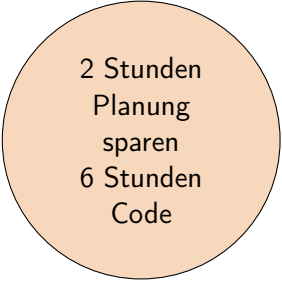
- ❶ **Frontend** – bedienbare Oberfläche, keine reine Konsole
- ❷ **Backend** – Logik in eigenen Modulen, getrennt vom Frontend
- ❸ **Datenquelle** – Datenbank (SQLite empfohlen) oder Excel
- ❹ **API** – mindestens eine externe REST-API
- ❺ **Datenverarbeitung** – pandas im Einsatz
- ❻ **Struktur und Doku** – modular, mit README und Docstrings

Zeit	Inhalt
Tag 0	Heute: Werkzeuge, Vorgehen, Setup
Tag 1	Anforderungsanalyse, Projektgerüst
Tag 2	API und Datenbank zum Laufen bringen
Tag 3	Erster Durchstich – alles verbunden
Tag 4	Pflichtfunktionen ausbauen
Tag 5	Robustheit, Datenschutz, Fehlerbehandlung
Tag 6	Polishing oder Kür
Tag 7	Dokumentation, Abgabe

**Wer ohne Plan loscodet,
landet ohne Plan irgendwo.**

Anforderungsanalyse ist:

- Klärung mit sich selbst, was man eigentlich bauen will
- Grundlage für sinnvolle Designentscheidungen
- Erste Übung im FIAE-Arbeitsstil



2 Stunden
Planung
sparen
6 Stunden
Code

Mindestens diese Abschnitte

- ① **Projektziel** – ein bis zwei Sätze
- ② **Funktionale Anforderungen** – was muss die Anwendung können?
- ③ **Nicht-funktionale Anforderungen** – wie schnell, wie zuverlässig, wie sicher?
- ④ **User Stories** – typische Nutzungssituationen
- ⑤ **Abgrenzung** – was soll die Anwendung *nicht* können?
- ⑥ **Risiken** – was könnte schiefgehen?
- ⑦ **Datenschutz** – welche Daten werden wie verarbeitet?

Vorlage als docx liegt im Begleitmaterial.

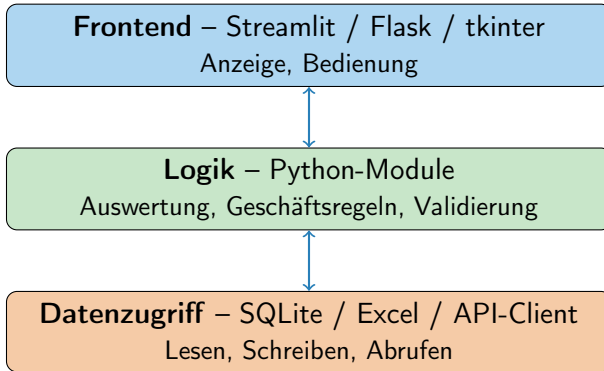
Variante B: Reise-Dashboard

Als jemand, der eine Städtereise plant,
will ich mehrere Wunschstädte in einer Liste verwalten,
damit ich ihre Wetterdaten vergleichen kann, ohne mehrere Webseiten parallel zu öffnen.

Format

“Als ... will ich ... damit ich ...”

Drei Bestandteile: **Rolle**, **Ziel**, **Nutzen**.



Warum diese Trennung?

- Logik testbar, ohne die Oberfläche zu öffnen
- Frontend austauschbar, ohne die Logik anfassen zu müssen
- Datenquelle austauschbar (SQLite heute, Postgres später)

Projektstruktur – ein Vorschlag

```
mein_projekt/  
+-- main.py  
+-- gui/  
|   +-- __init__.py  
|   +-- app.py           # Streamlit / Flask  
+-- core/  
|   +-- __init__.py  
|   +-- api_client.py    # API-Aufrufe  
|   +-- datenbank.py     # SQLite  
|   +-- auswertung.py    # pandas  
|   +-- modelle.py       # Datenklassen  
+-- daten/  
|   +-- meine_daten.db  
+-- .env                 # NICHT in Git!  
+-- .gitignore  
+-- requirements.txt  
+-- README.md
```

Sie werden in der Präsentation gefragt

“Warum haben Sie sich für SQLite und nicht für Excel entschieden?”

“Warum Streamlit und nicht Flask?”

“Warum diese Modulgliederung?”

Notieren Sie sich Ihre Gründe

Schon während des Projekts – nicht erst am letzten Tag. Eine kurze Notiz reicht: “Habe heute SQLite gewählt, weil ...”

Warum venv?

Jedes Projekt bekommt seine eigenen Bibliotheken. Keine Konflikte zwischen Projekten, keine “Funktioniert bei mir, aber nicht beim Dozenten”.

Schritt für Schritt

- ❶ `python -m venv .venv`
- ❷ Aktivieren:
 - Linux/Mac: `source .venv/bin/activate`
 - Windows: `.venv\Scripts\activate`
- ❸ `pip install streamlit requests pandas`
- ❹ `pip freeze > requirements.txt`

Werkzeug	Stärke	Aufwand	Empfehlung
Streamlit	Dashboard in Minuten, perfekt für Datenvisualisierung	sehr gering	Einstieg
Flask	Echte Webanwendung, eigene Routen, HTML-Templates	mittel	Kür
FastAPI	Moderne Web-API, sehr schnell, viel "Magie"	mittel-hoch	Kür
tkinter	Klassische Desktop-GUI, in Python eingebaut	mittel	Wenn Desktop gewünscht


```
# datei: app.py
import streamlit as st

st.title("Mein erstes Dashboard")

name = st.text_input("Wie heisst die Stadt?")

if st.button("Anzeigen"):
    st.write(f"Sie haben {name} eingegeben.")
    st.success("Daten erfolgreich uebernommen.")
```

Starten

```
streamlit run app.py
```

Öffnet automatisch den Browser auf localhost:8501.

```
# datei: app.py
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/anzeigen", methods=["POST"])
def anzeigen():
    stadt = request.form.get("stadt")
    return f"Eingegeben: {stadt}"

if __name__ == "__main__":
    app.run(debug=True)
```

Starten

`python app.py` – öffnet `localhost:5000`.
Erfordert HTML-Templates im Ordner `templates/`.

SQLite

- In Python eingebaut (`sqlite3`)
- Eine Datei, kein Server
- Echtes SQL
- Skaliert für dieses Projekt locker
- **Empfehlung**

Excel

- Bibliothek `openpyxl`
- Direkt in Excel öffnbar
- Kein SQL, keine Relationen
- Eher für Tabellen-Export geeignet
- Als Datenquelle nur, wenn es passt

Tipp

SQL-Kenntnisse sind im Beruf Standard. Wer noch nie mit SQL gearbeitet hat: Jetzt ist eine gute Gelegenheit, einzusteigen.

```
import sqlite3

verbindung = sqlite3.connect("daten.db")
cursor = verbindung.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS staedte (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        land TEXT,
        angelegt_am TEXT
    )
""")

cursor.execute(
    "INSERT INTO staedte (name, land, angelegt_am)
    VALUES (?, ?, ?)",
    ("Lissabon", "Portugal", "2026-01-15")
)
```

REST-APIs – was ist das?

Grundidee

Ein **REST-API** ist ein Dienst im Internet, der über HTTP angesprochen wird. Ihre Anwendung schickt eine Anfrage an eine URL, und der Dienst antwortet mit Daten – meist im JSON-Format.

Beispiel: Open-Meteo

```
https://api.open-meteo.com/v1/forecast?  
latitude=52.52&longitude=13.41&current=temperature_2m
```

Antwort (verkürzt):

```
{ "current": { "temperature_2m": 4.7 } }
```

Wichtig

APIs sind im Internet erreichbar – also können sie ausfallen. Ihre Anwendung muss damit umgehen.

API ansprechen mit requests

```
import requests

URL = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": 52.52,
    "longitude": 13.41,
    "current": "temperature_2m,wind_speed_10m"
}

try:
    antwort = requests.get(URL, params=params, timeout=5)
    antwort.raise_for_status()
    daten = antwort.json()
    print(daten["current"])
except requests.exceptions.Timeout:
    print("API antwortet nicht.")
except requests.exceptions.RequestException as fehler:
    print(f"Fehler beim API-Aufruf: {fehler}")
```


Was ist pandas?

Eine Bibliothek für tabellarische Daten. Zentrale Datenstruktur: der **DataFrame** – man kann sich vorzustellen wie eine Tabelle mit Spalten und Zeilen, ähnlich einer Excel-Tabelle.

Was Sie für das Projekt brauchen

- DataFrames aus Listen, Dicts, SQLite oder Excel erzeugen
- Spalten auswählen, Zeilen filtern
- Werte gruppieren und aggregieren (groupby)
- Einfache Berechnungen über Spalten
- Sortieren
- In SQLite zurückschreiben oder als CSV/Excel exportieren

```
import pandas as pd

daten = [
    {"stadt": "Berlin",    "temp": 4.5},
    {"stadt": "Hamburg",   "temp": 5.1},
    {"stadt": "Muenchen",  "temp": 2.8},
    {"stadt": "Berlin",    "temp": 5.0},
]

df = pd.DataFrame(daten)

# Mittelwert pro Stadt
durchschnitt = df.groupby("stadt")["temp"].mean()
print(durchschnitt)

# Filterung
warme = df[df["temp"] > 4]
print(warme)
```

Vier Mindestpunkte

- ❶ **Keine Geheimnisse im Code** – API-Schlüssel und Passwörter in `.env`, `.env` per `.gitignore` ausschließen.
- ❷ **Eingaben validieren** – Werte prüfen, bevor sie weiterverarbeitet werden.
- ❸ **Datensparsamkeit** – nur speichern, was Sie wirklich brauchen.
- ❹ **Datenflüsse dokumentieren** – in der Anforderungsanalyse beschreiben, welche Daten wo verarbeitet werden.

Im FIAE-Alltag normal

Datenschutz ist nicht “Bürokratie”, sondern Pflichtteil professioneller Softwareentwicklung. Sie üben das hier zum ersten Mal – es wird Sie Ihr Berufsleben begleiten.

Geheimnisse aus dem Code halten

```
# datei: .env (nicht ins Repository!)
API_KEY=geheimer_schluessel_12345
DATENBANK_PFAD=./daten/meine_daten.db
```

```
# datei: konfiguration.py
from dotenv import load_dotenv
import os

load_dotenv()

API_KEY = os.getenv("API_KEY")
DB_PFAD = os.getenv("DATENBANK_PFAD")
```

```
# datei: .gitignore
.env
.venv/
__pycache__/
*.db
```

Drei Fehlerkategorien

❶ API-Fehler

Keine Verbindung, Timeout, Server-Fehler, ungültige Antwort

❷ Datei- und Datenbank-Fehler

Datei nicht da, gesperrt, Datenbank korrupt, SQL-Fehler

❸ Eingabe-Fehler

Leere Felder, falsche Datentypen, unsinnige Werte

Verständliche Fehlermeldungen

Nicht: `KeyError: 'temperature_2m'`

Sondern: “Die Wetter-API hat keine Temperatur zurückgegeben. Bitte später erneut versuchen.”

Erlaubt und erwünscht

- Recherche und Verständnisfragen
- Syntax-Hilfe (“Wie funktioniert groupby?”)
- Fehlersuche (“Was bedeutet dieser Fehler?”)
- Code-Review nach dem Schreiben
- Alternativen vergleichen

Problematisch

- “Schreib mir das gesamte Projekt”
- Code übernehmen, ohne ihn zu verstehen
- KI nutzen, um Lernarbeit zu vermeiden statt zu erleichtern

Merksatz: Vor dem Übernehmen von KI-Code – diese drei Fragen mit “Ja”:

- 1 Verstehe ich, was dieser Code tut – Zeile für Zeile?
- 2 Könnte ich diesen Code in zwei Wochen einer Kollegin erklären?
- 3 Passt dieser Code zu meinem Projekt, oder ist er nur “ungefähr richtig”?

Warum dieser Test?

Sie werden in der Präsentation nach Designentscheidungen gefragt. Wer Code übernommen hat, ohne ihn zu verstehen, kommt dort schnell ins Schwimmen.

Einzelarbeit

Sie arbeiten an einem eigenen Projekt. Austausch über Konzepte und Vorgehensweisen ist erwünscht. Code-Sharing nicht.

Optionales Code-Review

Auf freiwilliger Basis dürfen Sie paarweise Code-Reviews machen – wenn beide das wollen und die Zeit dafür da ist.

Sinnvolle Zeitpunkte: Ende Tag 3 (erster Durchstich) und Ende Tag 5 (Pflicht im Wesentlichen fertig).

ZIP-Archiv mit...

- Quellcode in der entworfenen Projektstruktur
- `requirements.txt`
- `README.md`
- Anforderungsanalyse (überarbeitet)
- Projektdokumentation – Designentscheidungen, Stärken/Schwächen
- Beispiel-Output (Datenbank-Datei, Screenshots)
- **Keine Geheimnisse** (`.env`, API-Schlüssel)

Name

Nachname_Vorname_Projekt1.zip

Was Sie zeigen

- ➊ Kurzvorstellung (1 Min)
- ➋ Live-Demo (2–3 Min) – echte Bedienung
- ➌ Code-Walkthrough (2–3 Min) – eine interessante Stelle
- ➍ Designentscheidungen begründen (2 Min)
- ➎ Stärken, Schwächen, Potenziale (1–2 Min)

Demo-Backup

Screenshots oder eine kurze Bildschirmaufnahme vorbereiten. Falls am Präsentationstag etwas nicht läuft, sind Sie nicht aufgeschmissen.

- **Pflichtumsetzung** (40%) – alle sechs Pflichtpunkte, Anwendung läuft stabil
- **Code-Qualität** (25%) – Lesbarkeit, Struktur, Modularisierung
- **Dokumentation und Analyse** (15%) – Anforderungsanalyse passt zum Ergebnis
- **Präsentation und Reflexion** (20%) – Sie können Ihre Lösung erklären

Keine Noten

Sie bekommen konstruktives Feedback, keine Schulnoten. Ziel ist Lernen, nicht Bewertet-Werden.

❶ Variante wählen – A (Strom) oder B (Reise)

❷ Setup prüfen

- Python 3.10 oder höher installiert?
- IDE bereit (PyCharm, VS Code, andere)?
- Test mit `python --version`

❸ Erste Bibliotheken installieren

```
pip install streamlit requests pandas
```

❹ Mini-Hallo-Welt mit Streamlit

Das Streamlit-Beispiel von vorhin nachbauen und starten

❺ Erste API-Anfrage mit requests

Open-Meteo oder aWATTar einmal aufrufen, JSON ansehen

Erste Stunden

- Erst denken, dann codieren – Anforderungsanalyse zuerst.
- Klein anfangen. Erst die Verbindung zur API in einem winzigen Skript, dann erst in der Anwendung.
- Bei jedem Schritt: Geht das überhaupt? Hat die API geantwortet? Steht der Wert in der Datenbank?
- Wenn Sie nach zwei Stunden festhängen: *andere Person fragen*, nicht erst nach sechs Stunden.

Merksatz: Spätestens am Ende von Tag 3 sollte ein *laufender* Prototyp existieren – und sei er noch so einfach.

Im Begleitordner finden Sie

- **Projektaufgabe** (PDF) – die vollständige Aufgabenstellung
- **Anforderungsanalyse-Vorlage** (docx)
- **Cheat-Sheet** (PDF) – Quick-References zu Streamlit, SQLite, pandas, requests
- **Feedback-Raster** (PDF) – woran Sie bei der Selbstkontrolle messen können

Wann was lesen?

- Projektaufgabe und Vorlage **heute Abend** – damit Sie morgen früh starten können.
- Cheat-Sheet **Tag 2–3** – wenn Sie konkret an einer Stelle nicht weiterkommen.
- Feedback-Raster **Tag 5–6** – als Selbstcheck vor der Abgabe.

Fragen?

Jetzt ist Zeit fuer Ihre Fragen und Diskussionen

Los geht's!

Sieben Tage. Ein Projekt. Eine eigene Anwendung.

Variante wählen,
Setup machen,
morgen starten.